

ACIT4820 • Applied Robotics & Autonomous Systems

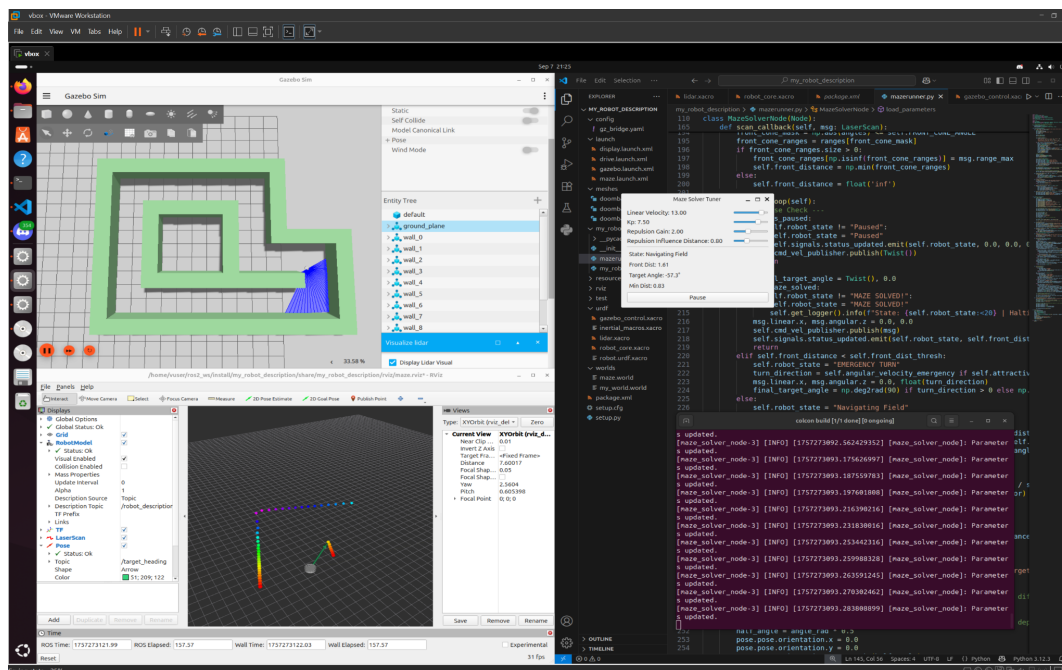
Assignment 3

Reactive maze navigation

Finalise the Doomba's body, then make it solve a maze on its own

Goal of this assignment

This assignment bridges teleoperation and autonomy. Building on your Assignment 2 robot, you'll finalise the Doomba's physical form with the provided 3D meshes and exact physical values. Then you'll implement an adaptive, reactive “sense-plan-act” loop that drives the robot through a maze and keeps working when the LiDAR's configuration changes underneath it. Along the way you'll measure how far your robot's own idea of where it is drifts from the truth. Every step asks for a drawing or a calculation as well as a screenshot. The drawings are where I see that you understood it.



What you're aiming for. Look closely: I tend to hide a hint or two in here.

Before you start

A working Assignment 2 robot (driving in Gazebo, LiDAR publishing `/scan`). You'll need the provided assets: `doomba_body.dae`, `doomba_wheel.dae`, `doomba_lidar.dae` and `maze.world`. Everything physical about the Doomba, its names, sizes, positions and masses, is on the next two pages; you'll use them all through Task 1.

The Doomba: names, sizes and masses

The Doomba is a small differential-drive robot (think a robot vacuum). It drives on two powered wheels set 3 cm behind the middle, balances on a **single passive caster sphere** at the front, and carries a LiDAR on top. Two driven wheels and one caster is three points of contact: the fewest a robot can stand on, and the most it can have before the load on each becomes a matter of opinion. Give it a fourth and it rocks.

One rename, now

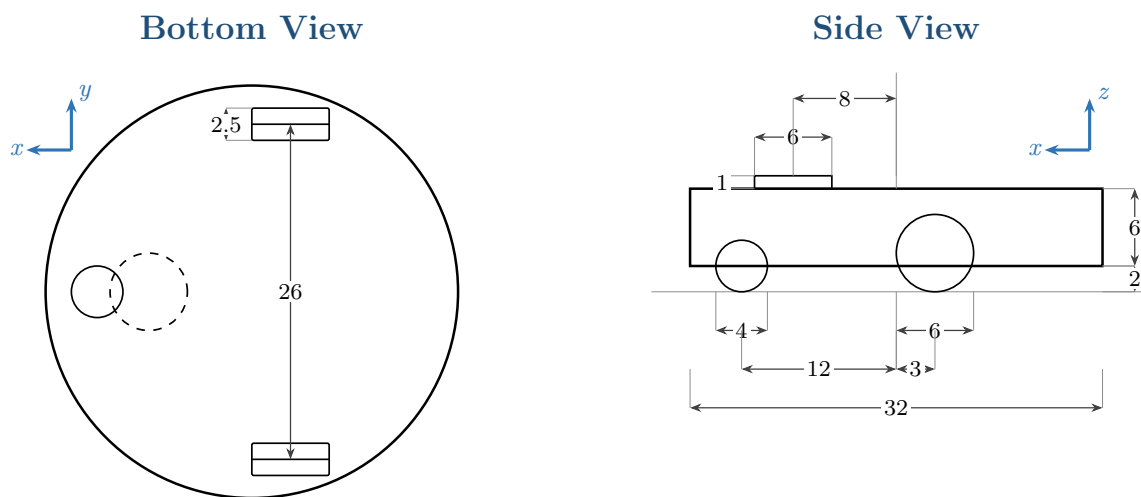
Use **exactly** the names below from now on. Your Assignment 1 and 2 robot may use others; this is the one place you change them, once, while you rebuild the URDF anyway: Assignment 4's SLAM and Nav2 configuration will be written against these names, and help in the room is far quicker when every robot agrees. A rename reaches further than the `<link>` and `<joint>` tags. Check every one of these:

- every `<gazebo reference="...">` block
- the diff-drive plugin's `<left_joint>` and `<right_joint>`, and its `<frame_id>` and `<child_frame_id>`
- the joint-state plugin's `<joint_name>` entries
- the LiDAR sensor's `<gz_frame_id>`
- the Fixed Frame and the LaserScan frame in your RViz config

A name that is wrong in one of these places fails in silence.

Link	Mass	Collision	Visual, and notes
<code>base_footprint</code>	(none)	(none)	Root of the TF tree; a virtual frame on the ground, no visual or inertial.
<code>base_link</code>	5.0 kg	cylinder	<code>doomba_body.dae</code> . The chassis.
<code>left_wheel_link</code>	0.2 kg	cylinder	<code>doomba_wheel.dae</code> . Powered.
<code>right_wheel_link</code>	0.2 kg	cylinder	<code>doomba_wheel.dae</code> . Powered.
<code>caster_wheel_link</code>	0.1 kg	sphere	sphere primitive; there is no caster mesh.
<code>lidar_link</code>	0.1 kg	cylinder	<code>doomba_lidar.dae</code> . 2D LiDAR on top.

Joint	Type	<origin xyz>	Where that comes from
<code>base_joint</code>	fixed	0 0 0.05	<code>base_footprint</code> to <code>base_link</code> : chassis centre 5 cm up
<code>left_wheel_joint</code>	continuous	-0.03 0.13 -0.02	3 cm aft, half the 26 cm track, axle 3 cm up
<code>right_wheel_joint</code>	continuous	-0.03 -0.13 -0.02	mirror of the left
<code>caster_wheel_joint</code>	fixed	0.12 0 -0.03	12 cm forward, sphere centre 2 cm up
<code>lidar_joint</code>	fixed	0.08 0 0.035	8 cm forward, sensor centre 8.5 cm up



The Doomba dimension diagram, in *centimetres*: bottom view (left) and side view (right). Body diameter 32, wheel separation 26, body height 6, ground clearance 2, wheels 3 behind the centre, caster 12 ahead, LiDAR 8 ahead.

URDF and Gazebo work in metres, so convert: a 32 cm body is a radius of 0.16 m.

Where the scan plane is

The LiDAR is 1 cm tall and sits on the roof, so its scan plane is 8.5 cm above the ground, half a centimetre above the chassis. Your model is rigid, so that gap never changes: get it right once and it stays right. Draw the body's collision cylinder exactly 6 cm tall, not a little taller to be safe, or the rear arc reads your own chassis at 16 cm in every scan, forever. The maze walls are 1 m tall, so the plane sees them everywhere.

Task 1: Finalise the Doomba's body and physics

Now we make your robot look and behave like the real Doomba: swap the primitives for meshes and pin down its exact physical description. Work in this order, since later steps depend on earlier ones.

- **Add the 3D meshes:** create a `meshes/` subfolder and place `doomba_body.dae`, `doomba_wheel.dae` and `doomba_lidar.dae` there. In your Xacro, replace the primitive geometry in the `<visual>` tags of `base_link`, both wheels and `lidar_link` with `<mesh>` tags pointing at these files. Add `meshes/` to your `setup.py data_files`, as you did for `launch/` and `urdf/` in Assignment 1, so they install. Rebuild. The caster keeps its `<sphere>` primitive for both `<visual>` and `<collision>`; it has no mesh.
- **Set the exact physical properties:** a simulation is only as good as its physical description. Take every size and position from the diagram and the joint origins table, and the masses from the link table. Put the dimensions in Xacro properties and derive the rest from them. A number that appears twice in a file will disagree with itself one day.
- **Compute the inertia matrices, by hand:** define an `<inertia>` for each link with mass, computed from its primitive collision shape with the formulas below. **Show the working in your report:** for each link the shape, the mass, the formula, the numbers you put in, and the result with its unit. Mind which axis the symmetry term lands on for the wheels.
- **Wire the physics to your Xacro properties:** in the diff-drive plugin, `<wheel_separation>` and `<wheel_radius>` must use the properties you defined from the diagram, not hard-coded copies.
- **Verify the collision model:** your robot's physics come from its `<collision>` tags, not its visuals. Reuse `display.launch.xml` from Assignment 1. Enable Collision in the RobotModel display and lower the model's Alpha so the collision shapes show through the meshes. **Include a screenshot** of this overlay view.
- **Verify the frames:** with the model running, read two transforms back from the tree and check them against the joint origins table, to the millimetre:

```
ros2 run tf2_ros tf2_echo base_link lidar_link
ros2 run tf2_ros tf2_echo base_link left_wheel_link
```

A wrong offset here never announces itself. It just smears the map in Assignment 4.

The three inertia formulas you need

All about each link's own centre of mass, off-diagonal terms zero; pair each with an **<inertial> <origin>** at that centre.

Solid cylinder, radius r , height h , round axis along z (base, wheels, LiDAR):

$$I_{xx} = I_{yy} = \frac{1}{12}m(3r^2 + h^2), \quad I_{zz} = \frac{1}{2}m r^2$$

Solid sphere, radius r (the caster):

$$I_{xx} = I_{yy} = I_{zz} = \frac{2}{5}m r^2$$

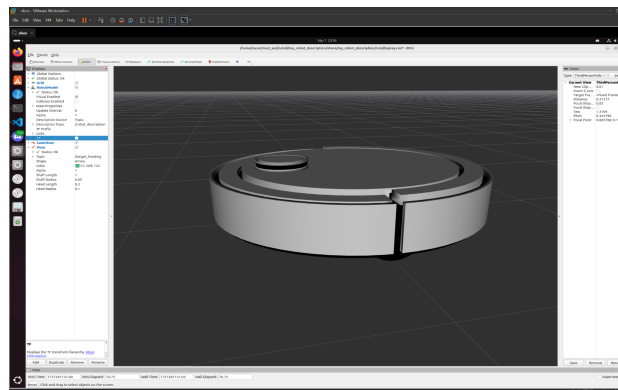
Solid box, sides a (x), b (y), c (z), for your project robot later:

$$I_{xx} = \frac{1}{12}m(b^2 + c^2), \quad I_{yy} = \frac{1}{12}m(a^2 + c^2), \quad I_{zz} = \frac{1}{12}m(a^2 + b^2)$$

Watch the axis. The cylinder formula puts $\frac{1}{2}mr^2$ on the round axis, assumed to be z . Your wheels spin about their round axis, which you rotated to point along the robot's y . Put the $\frac{1}{2}mr^2$ term on the wheel's *spin* axis in the link's own frame, never blindly on z . Worked wheel: $m = 0.2$ kg, $r = 0.03$ m, $w = 0.025$ m gives $\frac{1}{2}mr^2 \approx 9.0 \times 10^{-5}$ on the spin axis and $\frac{1}{12}m(3r^2 + w^2) \approx 5.5 \times 10^{-5}$ on the other two, in kg m². Doomba values live between 10^{-5} and 10^{-1} ; anything bigger is a slipped decimal or a centimetre that was never converted.

Where the mesh file lives, as far as Gazebo is concerned

A **package://my_robot_description/meshes/doomba_body.dae** URI works in RViz as soon as the package is installed. Gazebo resolves the same URI only if **GZ_SIM_RESOURCE_PATH** includes your workspace's **install/my_robot_description/share** folder; set it in your launch file or your shell. The alternative, **file://\$(find my_robot_description)/meshes/doomba_body.dae**, works in both without any setup, because xacro expands **\$(find ...)** to the full path before Gazebo ever sees it.



The finished, meshed Doomba you're building toward.

Task 2: The maze challenge

Now replace the keyboard controller with a Python node of your own that drives the robot through a maze on its own. Reference: [the LaserScan message](#).

- **The environment:** use the provided `maze.world`.
- **Draw before you code:** on paper, draw the robot beside a wall and the LiDAR beams you intend to use. Mark the distances you will read, the quantity you will compute from them (the error), and what the robot does with it (how the turn rate follows the error). This **controller diagram** goes in the report. If you cannot draw it, you cannot debug it.
- **Create `mazerunner.py`:** a node that solves the maze. A simple wall-follower is a good starting point, but you'll likely need to build on it with your own ideas to handle the harder cases below. Record a short **video** of a full run for the report.
- **Start paused:** a robust node does not drive the instant it launches, when Gazebo may still be loading. Give your node a small state machine that starts in a *paused* state and moves only after an unpaused command. Make it a ROS service, for example `/toggle_pause`, so you can call it from the command line. Paused means *publishing a zero Twist*, not publishing nothing: the diff-drive plugin keeps the last command it received, so a node that falls silent leaves the robot driving. You'll use this pause service in Task 4 to freeze the robot and read its state. Draw the **state machine** for the report: states, and what moves the robot between them.
- **Start simple:** begin with a LiDAR set to a **180°** field of view and **45** rays to get your algorithm working, then push from there.

Make the FOV a parameter

Task 3 asks you to test several LiDAR configurations. Rather than hand-editing the model each time, expose the LiDAR's **field of view** and **ray count** as Xacro arguments. Then you switch configurations from one place: cleaner, and good practice.

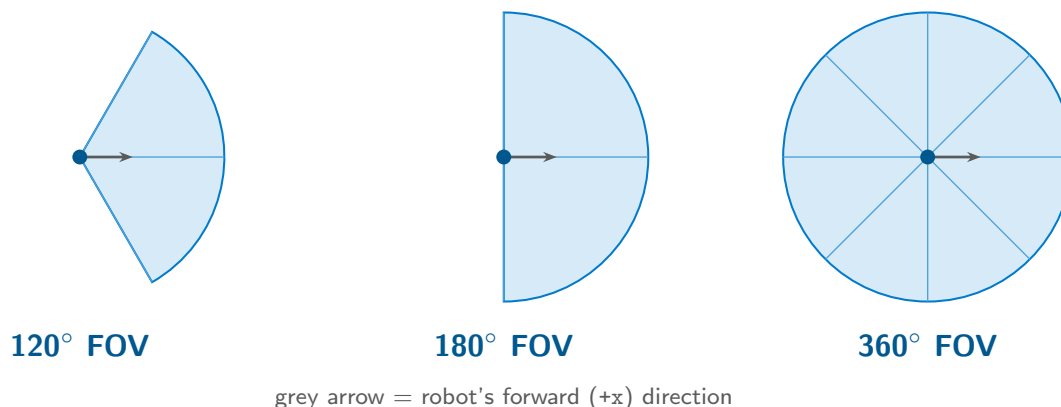
Your nose is wider than your body

The body radius is 16 cm, but the wheels sit 3 cm behind the centre, so when the robot spins on the spot its nose sweeps a circle of radius 19 cm. Spin beside a wall at 17 cm and the nose finds it. Choose the wall distance your controller holds with that number in mind, and **say in the report why you chose it**.

Task 3: Make the node adaptive and configurable

A well-written robotics node shouldn't break when a sensor's configuration changes. Refactor `mazerunner.py` to be robust to LiDAR changes.

- **Adaptive processing:** derive your logic from the LaserScan message's own fields (`angle_min`, `angle_max`, `angle_increment`, `ranges`), never from hard-coded indices. Unreturned rays come back as `inf` (or NaN), so guard against those before you use a distance.
- **Predict, then test:** before you run each configuration below, write down what you expect to break and why. Then run it, and write down what happened. One table, three rows: *configuration*, *prediction*, *observation*, *what you changed*. The gap between the second and third columns is the most honest line in your report. Keep the ray spacing at 4° throughout, so the field of view is the only thing that changes:
 - **Restricted FOV (120° , 30 rays):** less information about the sides, so sharp turns get harder.
 - **Full FOV (360° , 90 rays):** lots of extra information from behind. A naive algorithm can get confused by a wall it has already passed and try to turn around, so yours should stay focused on the direction of travel.
- **Parameterise your launch file:** set the important variables (gains, speeds, sector widths, and so on) as ROS parameters from the launch file, not as constants in the code.



The three LiDAR fields of view your node must handle. The 360° scan adds returns from behind the robot, which a naive wall-follower can mistake for a new wall.

Challenge: a tuning GUI

If you want to push yourself, build a small `rqt_gui` plugin with sliders for your parameters and text fields for debug output. It makes tuning far faster and shows you exactly what your robot is “thinking”, and its play/pause button can drive the pause service. Optional, and not a substitute for anything above.

Task 4: Where does the robot think it is?

Your robot publishes `/odom`: its own estimate of where it has driven, integrated from the wheel speeds by the diff-drive plugin. Gazebo knows where the robot really is. In this task you measure the gap between the two, and explain it.

- **Measure:** spawn the robot with zero yaw, so only its position needs subtracting later. Let it drive for a minute or two (your mazerunner, or teleop), then freeze it with your pause service. Read what it believes and what is true:

```
ros2 topic echo /odom --once
gz model --list                # the name you gave -entity when spawning
gz model -m <your model name> -p
```

`/odom` starts at zero at the spawn point; Gazebo's pose starts at the spawn position, so subtract that first. Both report the heading as a quaternion; for a robot on a flat floor the yaw is $2 \cdot \text{atan2}(q_z, q_w)$. Report the two positions and headings, and the gap in x, y and yaw.

- **Explain:** the plugin integrates two equations, with wheel radius r , wheel separation L and wheel speeds ω_L, ω_R :

$$v = \frac{r(\omega_R + \omega_L)}{2}, \quad \omega = \frac{r(\omega_R - \omega_L)}{L}.$$

Read them as a diagnostic. A wheel radius wrong by five percent makes every distance *and* every turn wrong by five percent, in the same ratio. A separation wrong by five percent leaves distances alone and scales only the turns. The caster scrubbing sideways through every turn and the wheels slipping add error of their own. Using the equations, say which of these your numbers point at.

- **Repeat once:** run the measurement a second time. Drift that keeps its sign is a modelling error you can fix; drift that changes sign is slip you can only estimate. Say which you saw. This gap is the exact problem SLAM solves in Assignment 4, so keep the numbers.

Explaining your solution

This report is read for understanding, and the drawings and calculations in the checklist below are where I look for it. One of them stands above the rest: a single high-level **conceptual diagram** that captures the whole problem and your solution in one illustration, the robot, the maze, what it senses, what it decides and what it does. It is required. It does not have to be pretty and it does not have to be right at the first attempt; it has to be an attempt, and yours. If you can distil the scenario and its solution into one informative drawing, you've truly understood it.

Hand-drawn and photographed is welcome; so is draw.io or TikZ. What matters is that the drawing is yours. A diagram generated for you shows me nothing, and I will ask about it. A derivation shows its steps and its units. It's perfectly fine if you don't solve every edge case, but document your thinking: what you tried, why you thought it would work, and your analysis of why it didn't. That critical reflection matters just as much as a clean run. This is the heaviest of the four assignments, so aim for the top of the word range, about **2,000 words**: four tasks, their drawings and the drift numbers will not fit in 1,000.

Supporting micro-lectures

Backed by: **C11** (meshes & inertia), **T2** (frames & transforms), **T4** (LaserScan in depth), **T5** (sense-plan-act & wall-following), **C12** (parameters & reconfigurable nodes) and **T3** (differential-drive kinematics & odometry).

What to submit

- ☐ A **screenshot** of the collision-overlay view, and the two `tf2_echo` readings against the joint origins table
- ☐ The inertia **derivations**: shape, mass, formula, numbers and unit for every link with mass
- ☐ The **conceptual diagram**: the whole problem and your solution in one drawing
- ☐ The **controller diagram** and the **state machine** of `mazerunner.py`, and a **flowchart** of its decision logic
- ☐ The wall distance your controller holds, and **why**, given the 19 cm sweep
- ☐ The **prediction table** for the three LiDAR configurations, with what you changed after each
- ☐ The **odometry drift**: both poses, the gap, the equations, and which cause your numbers point at, over two runs
- ☐ A short **video** of the robot solving the maze on its own
- ☐ Key code snippets from `mazerunner.py`, above all the adaptive scan handling and the parameters your launch file sets

Your report and what I require

I cannot stress this enough: a report that is a wall of text with no code and no pictures does not show me that you did the work or that you understood it. Last year a few submissions were essentially a piece of paper: no screenshots, no code, no evidence. That will not be approved. Here is what I am actually looking for.

What every report must contain

- ☐ **Screenshots of your progress.** At minimum, a screenshot at each major step (the specific ones are listed in “What to submit” above). I want to *see* your robot, your terminal, your RViz/Gazebo windows: proof that it ran on your machine.
- ☐ **Code snippets of the important bits.** Show me the parts that matter (your node logic, your launch file, the key YAML/Xacro). Snippets are welcome and expected, but keep them to the parts that carry the idea, not the entire codebase.
- ☐ **A short reflection.** What did you try, what worked, what didn’t, and *why*? An honest analysis of a failure is worth just as much to me as a clean success.
- ☐ **Structure and length.** A short intro, your design choices and method, results with the evidence above, and the reflection. Aim for roughly **1,000 to 2,000 words**; I am not looking for 15,000. I want a well-structured report that conveys the needed information in a professional and elegant manner.

Channel your favourite science communicator. I should finish your report convinced that *you* understood what you built, not just that something compiled. Don’t worry if every edge case isn’t solved; I am looking for effort, understanding, and a willingness to learn. We have plenty more to cover before the semester is over.

Formatting

I strongly recommend writing your report in L^AT_EX (Overleaf is great for this and works well in groups). Here is some layout inspiration, including how to add code snippets cleanly: overleaf.com/read/gstrwxnkwnhy.

On using AI / LLMs

LLMs are allowed as a **co-pilot for research and debugging**, the same way you’d use Google, the docs, or a knowledgeable friend. What they are *not* for is doing all your work and writing your report for you. The whole point of these assignments is that *you* build an understanding of how these systems fit together; if you outsource that, you’ve outsourced the learning. So use AI to get unstuck, then make sure you can explain every line you submit. Please briefly declare where and how you used AI in your report; being upfront about it is part of good engineering practice.

“Approved” means: a genuine attempt at all the tasks, plus a report that meets the requirements above and shows me your thinking. Good luck! :)